

시티아이랩_도시 교통신호 최적화 시스템 - Java/Spring

강화학습 기반 교통신호 최적화 시스템에서 메타데이터 영속화 · 시뮬레이션 오케스트레이션 · 실시간 스트리밍 · 분석 리포팅을 담당하는 Java/Spring 중간 계층을 설계·구현했습니다.

1. 프로젝트 개요

항목	내용
도메인	도시 교통신호 강화학습 최적화 (시뮬레이션 기반)
담당 영역	Java/Spring 백엔드 (영속화 · 외부 연동 · 실시간 스트리밍 · 리포팅)
기간	2025.09 ~ 2025.12
팀 구성	Java 백엔드(본인) / Python 연산 서버 / Unity 시각화 클라이언트 / 관리자 웹

시스템의 핵심 가설

고처리량 연산(강화학습·교통 시뮬레이션)은 **Python**이, 영속화·외부 인터페이스·운영 도구는 **Java**가 담당합니다.

이 분리 덕분에 학습이 수십 분~수 시간 걸려도 운영 화면이 멈추지 않았고, 두 서버가 서로의 장애로부터 격리되었습니다.

2. 담당 역할 & 기여 요약

영역	기여
모델/최적화 요청 관리	학습·최적화 요청 오케스트레이션, AI/Custom 분리, JPA Specification 동적 필터
동적 테이블 연동	연산 서버가 런타임에 생성하는 결과 테이블을 JdbcTemplate로 조회·정리
실시간 스트리밍	시각화 클라이언트에 시뮬레이션 데이터를 고정 주기로 WebSocket 푸시
신호 도메인 변환	Ring × Phase 신호 계획 매핑, Net 녹색시간 계산
분석 리포팅	전/후 지표 집계 JOIN 쿼리 + Apache POI 다중 시트 Excel 생성
공통 인프라	표준 응답 포맷, 전역 예외 처리, 요청 추적(MDC), WebClient/WebSocket/CORS 설정

3. 기술 스택

Java 21 · Spring Boot 2.7 · Spring WebFlux (WebClient) · Spring WebSocket · JPA /
Hibernate · JdbcTemplate · PostgreSQL · Apache POI · Gradle · log4j2

4. 시스템 아키텍처



5. 핵심 기술 의사결정

각 결정을 문제 → 선택 → 근거로 정리했습니다.

5.1 장시간 작업은 비동기, 삭제는 동기

- **문제:** 학습·최적화는 분~시간 단위. 블로킹 호출은 서버 스레드를 그만큼 점유 → 동시 요청 시 스레드풀 고갈. 반면 삭제는 정리 순서가 들어지면 데이터가 깨짐.
- **선택:** 쓰기 요청은 `WebClient.subscribe()` 로 즉시 반환하고 콜백에서 DB 반영, 삭제는 `.block()` 으로 외부 정리 → 종속 데이터 → 메타 순서를 보장.

```
// 쓰기
webClient.post().uri(START_URI).bodyValue(req)
```

```

        .retrieve().bodyToMono(Resp.class)
        .subscribe(resp -> updateOnCallback(id, resp),
            err -> log.error("연산 서버 호출 실패", err));

// 삭제
Resp resp = webClient.delete().uri(DELETE_URI)
    .retrieve().bodyToMono(Resp.class).block();
cleanupDependents(id);
metaRepository.deleteById(id);

```

- **근거:** 프론트엔드 즉시성과 데이터 정합성을 요청 성격에 따라 분리. 한 단계라도 실패하면 후속 단계를 중단시켜 부분 삭제 상태를 방지.

5.2 런타임 동적 테이블은 JPA 대신 JdbcTemplate

- **문제:** 연산 서버가 시뮬레이션마다 이름이 다른 결과 테이블을 생성. JPA Entity는 컴파일 타임에 테이블명이 고정되어야 함 → 표현 불가.
- **선택:** 해당 영역만 `JdbcTemplate` 으로 런타임 테이블명을 조립해 조회/삭제. 정형 데이터는 JPA를 유지.
- **근거:** 도메인 요구가 ORM 범위를 벗어나는 지점만 선택적으로 JDBC로 내려, 나머지 영역의 ORM 안전성은 그대로 확보. (테이블명 조립부에는 식별자 검증을 추가하는 것을 개선 과제로 명시 — 8장 참고)

5.3 실시간 시각화는 고정 주기 스케줄링

- **문제:** 시각화 클라이언트가 부드러운 애니메이션을 요구하나, 매 프레임 DB 조회는 부하가 큼. 원본 시뮬레이션 데이터는 1초 단위.
- **선택:** `ScheduledExecutorService` 로 **150ms 고정 주기** 푸시. 33ms(30fps)는 초당 DB 쿼리 과다, 500ms 이상은 끊겨 보임 → 부하와 체감의 절충점으로 6~7fps 선택.
- **근거:** 데이터 해상도(1초)보다 자주 폴링해도 새 데이터가 없으므로, "충분히 부드럽되 과하지 않은" 주기를 정량적으로 근거를 두고 결정.

5.4 요청 추적 — MDC traceId 표준화

- **문제:** 클라이언트가 받은 응답만으로 서버 로그에서 해당 요청을 찾을 방법이 없음.

- **선택:** Servlet Filter에서 요청마다 UUID traceId를 MDC에 주입하고, 표준 응답 객체가 이를 자동 포함.

```
try {
    MDC.put("traceId", UUID.randomUUID().toString());
    chain.doFilter(request, response);
} finally {
    MDC.remove("traceId"); // 스레드 재사용 시 이전 요청 ID 오염 방지
}
```

- **근거:** 응답의 traceId → 서버 로그 grep으로 요청 전체 흐름 추적. `try-finally` 로 스레드풀 오염 차단.

5.5 N+1 제거 & 동적 필터

- **문제:** 목록 조회 시 행마다 연관 데이터를 조회하면 N+1. 또한 사용자가 임의 조합하는 검색 필터를 JPQL 문자열 분기로 짜면 가독성·안전성 저하.
- **선택:** 연관 키를 모아 `IN` 절로 배치 조회 후 Map 록업(쿼리 $O(N) \rightarrow O(1)$). 검색은 **JPA Specification**으로 값이 있을 때만 조건을 추가.

```
// N+1 제거
List<Key> keys = rows.stream().map(Row::getRefKey).distinct()
    .toList();
Map<Key, Ref> refMap = refRepo.findByRefKeyIn(keys).stream()
    .collect(toMap(Ref::getRefKey, r -> r));
return rows.stream().map(r -> toDto(r, refMap.get(r.getRefKey())))
    .toList();
```

- **근거:** 조회 성능 문제를 구조적으로 제거하고, 동적 쿼리는 타입 안전하게 조립.

6. 공통 계층 표준화

- **통합 응답 포맷:** 모든 REST 응답을 `record` 기반 `ResponseApi<T>` 하나로 통일. 생성 시점에 traceId·timestamp 자동 주입 → 불변 + 추적 가능.

- **비즈니스 코드와 HTTP 코드 분리:** 같은 HTTP 400이라도 "파라미터 형식 오류"와 "검증 실패"를 별도 비즈니스 코드로 구분해 프론트엔드가 다르게 처리 가능.
- **전역 예외 처리:** `@RestControllerAdvice` 로 비즈니스 예외는 메시지 그대로, 시스템 예외는 고정 메시지 + 500으로 변환해 내부 오류 상세의 외부 노출을 차단(공격 표면 축소), 서버 로그에만 전체 스택 기록.

7. 트러블슈팅 신호 데이터 충돌 — 조합 키로 해결

시범 운영 데이터에서 서로 다른 Ring이 동일한 phase 번호를 가져 `Collectors.toMap` 이 충돌(`IllegalStateException`)하는 문제 발생.

```
// 키를 (ringType + phaseNo) 조합으로 만들어 충돌 회피 + 머지 함
수로 방어
.collect(Collectors.toMap(
    p -> p.getRingType() + "-" + p.getPhaseNo(),
    p -> p.getOpTime(),
    (a, b) -> a)); // 중복 데이터가 있어도 첫 값 유지
```

실제 운영 데이터의 모서리 케이스를 코드 레벨에서 방어적으로 해결하고, 데이터가 깨져도 화면은 깨지지 않도록 fallback을 설계.

리포팅 — 다중 시트 Excel

전/후 지표를 화면용 JSON과 보고서용 Excel 두 형태로 제공. Apache POI로 관점별 다중 시트(시간대별 지표 / 지점별 효과 / 신호 변경 / 안전 시간)를 생성하고, 시트별 헤더 색상을 도메인 의미와 일치시켜 비개발 검토자도 즉시 이해하도록 구성.

8. 한계 인식 & 개선 로드맵

도입 시점에 의도적으로 우선순위를 미룬 부분을 인지하고, 개선 방향까지 정리했습니다.

우선순위	항목	개선 방향
높음	인증·인가 미구현 (내부망 전제)	JWT/세션 기반 인증 도입
높음	외부 서버 주소·자격증명 외부화 필요	<code>@ConfigurationProperties</code> + 프로파일별 설정

우선순위	항목	개선 방향
중	동적 테이블명 식별자 검증	UUID 정규식 검증 추가
중	예외 처리 방식 이원화	전역 핸들러로 일원화
낮음	단일 세션 스트리밍	세션별 Map + 태스크 분리